

[OPTIONAL] Local Deployment of N8N Agentic Ecosystem

*Configuration Guide: Containerized N8N,
Qdrant, LiteLLM, Redis, PGSql, ...*

(Windows & macOS)

July 26, 2026

WARNING/DISCLAIMER

Please Note: These instructions are compiled based on what works on most of the High-end laptops/systems, including (but not limited to) 16GB RAM, 1TB DiskSpace, etc. However, please be advised that the instructions are for self-help and self-exploratory purposes, and there is no guaranty that the software discussed in this document will work hassle-free. For example, the installations may cause the laptop/device to slow down drastically, and/or bestow unwarranted results due to the open-source nature of the softwares. Please ensure adequate backups of your system before following this guide and at your own risk.

ALTERNATIVE

*Please Note: Use of this install guide is **COMPLETELY OPTIONAL**. As an alternative, please use online / cloud subscriptions of N8N, Postgres, etc.*

Contents

1	Introduction & Ecosystem Architecture	3
2	System Prerequisites	3
2.1	Hardware Requirements	3
2.2	Software Dependencies	3
3	Pre-Installation Configurations	4
3.1	Windows Configuration (WSL 2)	4
3.2	macOS Configuration	4
4	Deploying the Ecosystem	4
4.1	Step 1: Directory & Volume Preparation	4
4.2	Step 2: Environment Variables (.env)	5
4.3	Step 3: Creating the docker-compose.yml	5
4.4	Step 4: Executing the Deployment	7

5 Verification & Next Steps

7

1 Introduction & Ecosystem Architecture

This guide provides step-by-step instructions for deploying a comprehensive agentic AI ecosystem on a single laptop. The stack leverages Docker Compose to orchestrate multiple services efficiently without conflicting with host operations.

Core Components:

- **n8n (Port 5678):** The orchestrator and primary workflow engine for designing agentic pathways.
- **Qdrant (Ports 6333, 6334):** A high-performance vector database for Retrieval-Augmented Generation (RAG).
- **LiteLLM Proxy (Port 4000):** Unified interface for routing API calls to underlying LLMs and logging metadata to Langfuse.
- **PostgreSQL (Port 5432):** Relational database used for n8n memory persistence and LiteLLM routing rules.

2 System Prerequisites

To ensure smooth operation of multiple AI containers (especially when interacting with local LLMs via Ollama), the following system capabilities are required:

2.1 Hardware Requirements

- **CPU:** Quad-core processor minimum (Apple Silicon M1/M2/M3 for macOS, Intel i5/i7 10th Gen+ or AMD Ryzen 5+ for Windows).
- **RAM:** 16 GB minimum (32 GB highly recommended if running local LLMs like Llama-3 concurrently).
- **Storage:** At least 20 GB of free SSD space for Docker images, vector storage, and databases.

2.2 Software Dependencies

Before starting, ensure the following tools are installed:

- **Docker Desktop:** The containerization engine.
- **Git:** For repository cloning and version control.
- **Code Editor:** Visual Studio Code, PyCharm, or similar (for editing `.env` and YAML files).

3 Pre-Installation Configurations

3.1 Windows Configuration (WSL 2)

Windows runs Docker via the Windows Subsystem for Linux (WSL 2). It is critical to allocate sufficient resources.

1. Install WSL 2 by opening PowerShell as Administrator and running: `wsl --install`.
2. Restart your machine.
3. Install **Docker Desktop for Windows**. During installation, ensure the *"Use WSL 2 instead of Hyper-V"* option is checked.

4. **Resource Allocation:** By default, WSL 2 dynamically allocates RAM. To prevent Out-Of-Memory (OOM) errors, create a `.wslconfig` file in your user directory (`C:\Users\YourName\.wslconfig`):

```
[wsl2]
memory=12GB
processors=4
swap=4GB
```

5. Open PowerShell and run `wsl --shutdown` to apply changes, then restart Docker Desktop.

3.2 macOS Configuration

Docker Desktop on macOS runs within a lightweight virtual machine.

1. Download and install **Docker Desktop for Mac** (ensure you select the correct version: Apple Silicon vs. Intel).
2. Open Docker Desktop → Click the **Settings (Gear)** icon → Navigate to **Resources**.
3. **Resource Allocation:** Increase the limits to prevent crashes.
 - **CPUs:** 4 or more.
 - **Memory:** 12 GB or more.
 - **Virtual Disk Limit:** Set to at least 40 GB.
4. Click **Apply & Restart**.

4 Deploying the Ecosystem

4.1 Step 1: Directory & Volume Preparation

Create a dedicated folder for your project and set up the necessary subdirectories for persistent data mapping.

Windows (PowerShell):

```
mkdir C:\n8n-agentic-stack
cd C:\n8n-agentic-stack
mkdir n8n_data, qdrant_data
```

macOS (Terminal):

```
mkdir ~/n8n-agentic-stack
cd ~/n8n-agentic-stack
mkdir n8n_data qdrant_data
```

4.2 Step 2: Environment Variables (.env)

Create a file named `.env` in your project root directory. This securely holds your API keys and configuration settings.

```
# .env file
GENERIC_TIMEZONE=Asia/Kolkata
TZ=Asia/Kolkata

# PostgreSQL Settings
POSTGRES_USER=ai_user
POSTGRES_PASSWORD=ai_password
POSTGRES_DB=ai_database

# Langfuse / LiteLLM Config
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_HOST=http://host.docker.internal:3000
```

Note: The `host.docker.internal` string allows LiteLLM to route traffic properly to locally hosted models (e.g., Ollama) or locally hosted Langfuse instances on both Windows and Mac.

4.3 Step 3: Creating the docker-compose.yml

Create a `docker-compose.yml` file in the same directory. This file defines the orchestrator, vector DB, and proxy.

```
version: '3.8'

services:
  # 1. ORCHESTRATOR
  n8n:
    image: docker.n8n.io/n8nio/n8n
    container_name: n8n
    ports:
      - "5678:5678"
    environment:
      - GENERIC_TIMEZONE=${GENERIC_TIMEZONE}
      - TZ=${TZ}
      - N8N_HOST=0.0.0.0
      - N8N_PORT=5678
      - N8N_PROTOCOL=http
      - NODE_ENV=production
      - WEBHOOK_URL=http://localhost:5678/
    volumes:
      - n8n_data:/home/node/.n8n
```

```
# Map your local workflow policies here
- ./policies:/home/node/.n8n-files/policies
networks:
- ai_net
restart: unless-stopped

# 2. VECTOR DATABASE
qdrant:
  image: qdrant/qdrant
  container_name: qdrant_vectordb
  ports:
    - "6333:6333" # REST API & Web UI
    - "6334:6334" # gRPC
  volumes:
    - qdrant_data:/qdrant/storage
  networks:
    - ai_net
  restart: unless-stopped

# 3. MLOPS & ROUTING PROXY
litellm:
  image: ghcr.io/berriai/litellm:main-latest
  container_name: litellm_proxy
  ports:
    - "4000:4000"
  env_file:
    - .env
  environment:
    - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}
    @postgres:5432/${POSTGRES_DB}
  networks:
    - ai_net
  depends_on:
    - postgres
  restart: unless-stopped

# 4. MEMORY STORE
postgres:
  image: postgres:15
  container_name: postgres_db
  environment:
    - POSTGRES_USER=${POSTGRES_USER}
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - POSTGRES_DB=${POSTGRES_DB}
  ports:
    - "5432:5432"
  volumes:
    - pg_data:/var/lib/postgresql/data
  networks:
    - ai_net
  restart: unless-stopped

volumes:
  n8n_data:
  qdrant_data:
  pg_data:

networks:
```

```
ai_net:  
  driver: bridge
```

4.4 Step 4: Executing the Deployment

Once the files are prepared, spin up the entire ecosystem.

Windows (PowerShell):

```
docker-compose up -d
```

macOS (Terminal):

```
docker compose up -d
```

Docker will pull all necessary images and spin up the containers sequentially. The `-d` flag runs them in detached mode.

5 Verification & Next Steps

Once the containers are running, verify access by navigating to the following URLs in your web browser:

- **n8n Dashboard:** <http://localhost:5678> (Complete the initial admin account setup).
- **Qdrant Web UI:** <http://localhost:6333/dashboard> (Verify the visual dashboard loads).
- **LiteLLM Proxy:** <http://localhost:4000/health/liveness> (Should return a 200 OK status).

Integration Note: When configuring n8n AI nodes to connect to your local Ollama instance (or LiteLLM proxy), remember to use `http://host.docker.internal:4000` (or 11434 for Ollama) as the Base URL instead of `localhost`. This ensures n8n reaches outside its container to the host network correctly.